Python Assignment – 6

1) Write a function ‘mode_digit’ that takes an integer number ‘n’ and returns the digit that appears most frequently in that number. The given number may be positive or negative, but the most frequent digit returned should always be non-negative. If there is a tie, the digit with the greatest value should be returned.

```python
def digit_list(n):
    l = list()
    while n != 0:
        l.append(n%10)
        n //= 10
    return l
def mode_digit(n):
    l = digit_list(n)
    cl = list()
    for i in l:
        c = l.count(i)
        cl.append(c)
    m = max(cl)
    x = -1
    for i in l:
        if l.count(i) == m:
            if i>x:
                x = i
    return x
n = int(input('Enter a Number: '))
print(f'The digit which appears the most in {n} is {mode_digit(abs(n))}.')
```

Run: Rough Work ×

```
C:\Users\Yuvi\PycharmProjects\pythonProject\venv\Scripts\python.exe "C:/Users/Yu
Enter a Number: -988766
The digit which appears the most in -988766 is 8.
```

2) Write a Python program to replace all the even elements by 0 and odd elements by 1 in the given list.

```python
n = int(input('No. of Elements in the List: '))
l = [0]*n
for i in range(n):
    l[i] = int(input('Element {}: '.format(i+1)))
print('Original List:',l)
for i in range(n):
    if l[i] % 2 == 0:
        l[i] = 0
    else:
        l[i] = 1
print('Modified List:',l)
```

Run: Rough Work ×

```
C:\Users\Yuvi\PycharmProjects\pythonProject\venv\Sc
No. of Elements in the List: 5
Element 1: 1
Element 2: 7
Element 3: 9
Element 4: 133
Element 5: 22
Original List: [1, 7, 9, 133, 22]
Modified List: [1, 1, 1, 1, 0]
```

3) Given a list of tuples containing co-ordinates, write a Python program to find which co-ordinate is the closest to a co-ordinate given in input.

```python
from math import pow
l = list()
print('Obtaining the random co-ordinates...')
while True:
    x = int(input('x co-ordinate: '))
    y = int(input('y co-ordinate: '))
    l.append((x,y))
    z = input('Continue? (y/n): ')
    if z == 'n':
        break
print('\nObtaining the main co-ordinates...')
x = int(input('x co-ordinate: '))
y = int(input('y co-ordinate: '))
d, a, b = pow((x-l[0][0])**2 + (y-l[0][1])**2,0.5), l[0][0], l[0][1]
for i in l:
    d1 = pow((x-i[0])**2 + (x-i[1])**2,0.5)
    if d1 < d:
        d = d1
        a,b = i[0],i[1]
print(f'\nThe point closest to {(x,y)} is {(a,b)}. The distance between these two points is {d:.2f} units.')
```

Rough Work ×

```
C:\Users\Yuvi\PycharmProjects\pythonProject\venv\Scripts\python.exe "C:/Users/Yuvi/PycharmProjects/pythonProject/venv/Rough Work.py"
Obtaining the random co-ordinates...
x co-ordinate: 6
y co-ordinate: 9
Continue? (y/n): y
x co-ordinate: 3
y co-ordinate: 4
Continue? (y/n): n

Obtaining the main co-ordinates...
x co-ordinate: 0
y co-ordinate: 0

The point closest to (0, 0) is (3, 4). The distance between these two points is 5.00 units.
```

4) Consider the following 2 sets

x = {x: x is a natural number and a factor of 1024}
y = {x: x is a natural even number less than 1200}

The program should print the result of set operations such as union, intersection, set difference, and symmetric difference.

```python
x = set([x for x in range(1,1025) if 1024 % x == 0])
y = set([x for x in range(2,1200,2)])
print(f'x = {x}')
print(f'y = {y}')
print(f'\nUnion: {x|y}')
print(f'\nIntersection: {x&y}')
print(f'\nSet Difference: {x-y}')
print(f'\nSymmetric Difference: {x^y}')
```

```
x = {32, 1, 2, 64, 4, 128, 256, 512, 8, 1024, 16}
y = {2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 42, 44, 46, 48, 50, 52, 54, 56, 58, 60, 62, 64, 66, 68, 70, 72, 74, 76, 78, 80, 82, 84, 86, 88, 90, 92, 94, 96, 98, 100, 102, 104, 106, 108, 110, 112, 114, 116, 118, 120, 122,
 124, 126, 128, 130, 132, 134, 136, 138, 140, 142, 144, 146, 148, 150, 152, 154, 156, 158, 160, 162, 164, 166, 168, 170, 172, 174, 176, 178, 180, 182, 184, 186, 188, 190, 192, 194, 196, 198, 200, 202, 204, 206, 208, 210, 212, 214, 216, 218, 220, 222, 224,
 226, 228, 230, 232, 234, 236, 238, 240, 242, 244, 246, 248, 250, 252, 254, 256, 258, 260, 262, 264, 266, 268, 270, 272, 274, 276, 278, 280, 282, 284, 286, 288, 290, 292, 294, 296, 298, 300, 302, 304, 306, 308, 310, 312, 314, 316, 318, 320, 322, 324, 326,
 328, 330, 332, 334, 336, 338, 340, 342, 344, 346, 348, 350, 352, 354, 356, 358, 360, 362, 364, 366, 368, 370, 372, 374, 376, 378, 380, 382, 384, 386, 388, 390, 392, 394, 396, 398, 400, 402, 404, 406, 408, 410, 412, 414, 416, 418, 420, 422, 424, 426, 428,
 430, 432, 434, 436, 438, 440, 442, 444, 446, 448, 450, 452, 454, 456, 458, 460, 462, 464, 466, 468, 470, 472, 474, 476, 478, 480, 482, 484, 486, 488, 490, 492, 494, 496, 498, 500, 502, 504, 506, 508, 510, 512, 514, 516, 518, 520, 522, 524, 526, 528, 530,
 532, 534, 536, 538, 540, 542, 544, 546, 548, 550, 552, 554, 556, 558, 560, 562, 564, 566, 568, 570, 572, 574, 576, 578, 580, 582, 584, 586, 588, 590, 592, 594, 596, 598, 600, 602, 604, 606, 608, 610, 612, 614, 616, 618, 620, 622, 624, 626, 628, 630, 632,
 634, 636, 638, 640, 642, 644, 646, 648, 650, 652, 654, 656, 658, 660, 662, 664, 666, 668, 670, 672, 674, 676, 678, 680, 682, 684, 686, 688, 690, 692, 694, 696, 698, 700, 702, 704, 706, 708, 710, 712, 714, 716, 718, 720, 722, 724, 726, 728, 730, 732, 734,
 736, 738, 740, 742, 744, 746, 748, 750, 752, 754, 756, 758, 760, 762, 764, 766, 768, 770, 772, 774, 776, 778, 780, 782, 784, 786, 788, 790, 792, 794, 796, 798, 800, 802, 804, 806, 808, 810, 812, 814, 816, 818, 820, 822, 824, 826, 828, 830, 832, 834, 836,
 838, 840, 842, 844, 846, 848, 850, 852, 854, 856, 858, 860, 862, 864, 866, 868, 870, 872, 874, 876, 878, 880, 882, 884, 886, 888, 890, 892, 894, 896, 898, 900, 902, 904, 906, 908, 910, 912, 914, 916, 918, 920, 922, 924, 926, 928, 930, 932, 934, 936, 938,
 940, 942, 944, 946, 948, 950, 952, 954, 956, 958, 960, 962, 964, 966, 968, 970, 972, 974, 976, 978, 980, 982, 984, 986, 988, 990, 992, 994, 996, 998, 1000, 1002, 1004, 1006, 1008, 1010, 1012, 1014, 1016, 1018, 1020, 1022, 1024, 1026, 1028, 1030, 1032, 1034,
 1036, 1038, 1040, 1042, 1044, 1046, 1048, 1050, 1052, 1054, 1056, 1058, 1060, 1062, 1064, 1066, 1068, 1070, 1072, 1074, 1076, 1078, 1080, 1082, 1084, 1086, 1088, 1090, 1092, 1094, 1096, 1098, 1100, 1102, 1104, 1106, 1108, 1110, 1112, 1114, 1116, 1118,
 1120, 1122, 1124, 1126, 1128, 1130, 1132, 1134, 1136, 1138, 1140, 1142, 1144, 1146, 1148, 1150, 1152, 1154, 1156, 1158, 1160, 1162, 1164, 1166, 1168, 1170, 1172, 1174, 1176, 1178, 1180, 1182, 1184, 1186, 1188, 1190, 1192, 1194, 1196, 1198}

Union: {1, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 42, 44, 46, 48, 50, 52, 54, 56, 58, 60, 62, 64, 66, 68, 70, 72, 74, 76, 78, 80, 82, 84, 86, 88, 90, 92, 94, 96, 98, 100, 102, 104, 106, 108, 110, 112, 114, 116, 118, 120,
 122, 124, 126, 128, 130, 132, 134, 136, 138, 140, 142, 144, 146, 148, 150, 152, 154, 156, 158, 160, 162, 164, 166, 168, 170, 172, 174, 176, 178, 180, 182, 184, 186, 188, 190, 192, 194, 196, 198, 200, 202, 204, 206, 208, 210, 212, 214, 216, 218, 220, 222,
 224, 226, 228, 230, 232, 234, 236, 238, 240, 242, 244, 246, 248, 250, 252, 254, 256, 258, 260, 262, 264, 266, 268, 270, 272, 274, 276, 278, 280, 282, 284, 286, 288, 290, 292, 294, 296, 298, 300, 302, 304, 306, 308, 310, 312, 314, 316, 318, 320, 322, 324,
 326, 328, 330, 332, 334, 336, 338, 340, 342, 344, 346, 348, 350, 352, 354, 356, 358, 360, 362, 364, 366, 368, 370, 372, 374, 376, 378, 380, 382, 384, 386, 388, 390, 392, 394, 396, 398, 400, 402, 404, 406, 408, 410, 412, 414, 416, 418, 420, 422, 424, 426,
 428, 430, 432, 434, 436, 438, 440, 442, 444, 446, 448, 450, 452, 454, 456, 458, 460, 462, 464, 466, 468, 470, 472, 474, 476, 478, 480, 482, 484, 486, 488, 490, 492, 494, 496, 498, 500, 502, 504, 506, 508, 510, 512, 514, 516, 518, 520, 522, 524, 526, 528,
 530, 532, 534, 536, 538, 540, 542, 544, 546, 548, 550, 552, 554, 556, 558, 560, 562, 564, 566, 568, 570, 572, 574, 576, 578, 580, 582, 584, 586, 588, 590, 592, 594, 596, 598, 600, 602, 604, 606, 608, 610, 612, 614, 616, 618, 620, 622, 624, 626, 628, 630,
 632, 634, 636, 638, 640, 642, 644, 646, 648, 650, 652, 654, 656, 658, 660, 662, 664, 666, 668, 670, 672, 674, 676, 678, 680, 682, 684, 686, 688, 690, 692, 694, 696, 698, 700, 702, 704, 706, 708, 710, 712, 714, 716, 718, 720, 722, 724, 726, 728, 730, 732,
 734, 736, 738, 740, 742, 744, 746, 748, 750, 752, 754, 756, 758, 760, 762, 764, 766, 768, 770, 772, 774, 776, 778, 780, 782, 784, 786, 788, 790, 792, 794, 796, 798, 800, 802, 804, 806, 808, 810, 812, 814, 816, 818, 820, 822, 824, 826, 828, 830, 832, 834,
 836, 838, 840, 842, 844, 846, 848, 850, 852, 854, 856, 858, 860, 862, 864, 866, 868, 870, 872, 874, 876, 878, 880, 882, 884, 886, 888, 890, 892, 894, 896, 898, 900, 902, 904, 906, 908, 910, 912, 914, 916, 918, 920, 922, 924, 926, 928, 930, 932, 934, 936,
 938, 940, 942, 944, 946, 948, 950, 952, 954, 956, 958, 960, 962, 964, 966, 968, 970, 972, 974, 976, 978, 980, 982, 984, 986, 988, 990, 992, 994, 996, 998, 1000, 1002, 1004, 1006, 1008, 1010, 1012, 1014, 1016, 1018, 1020, 1022, 1024, 1026, 1028, 1030, 1032,
 1034, 1036, 1038, 1040, 1042, 1044, 1046, 1048, 1050, 1052, 1054, 1056, 1058, 1060, 1062, 1064, 1066, 1068, 1070, 1072, 1074, 1076, 1078, 1080, 1082, 1084, 1086, 1088, 1090, 1092, 1094, 1096, 1098, 1100, 1102, 1104, 1106, 1108, 1110, 1112, 1114, 1116, 1118,
 1120, 1122, 1124, 1126, 1128, 1130, 1132, 1134, 1136, 1138, 1140, 1142, 1144, 1146, 1148, 1150, 1152, 1154, 1156, 1158, 1160, 1162, 1164, 1166, 1168, 1170, 1172, 1174, 1176, 1178, 1180, 1182, 1184, 1186, 1188, 1190, 1192, 1194, 1196, 1198}

Intersection: {32, 64, 2, 128, 4, 256, 512, 1024, 8, 16}

Set Difference: {1}

Symmetric Difference: {1, 6, 10, 12, 14, 18, 20, 22, 24, 26, 28, 30, 34, 36, 38, 40, 42, 44, 46, 48, 50, 52, 54, 56, 58, 60, 62, 66, 68, 70, 72, 74, 76, 78, 80, 82, 84, 86, 88, 90, 92, 94, 96, 98, 100, 102, 104, 106, 108, 110, 112, 114, 116, 118, 120, 122,
 124, 126, 130, 132, 134, 136, 138, 140, 142, 144, 146, 148, 150, 152, 154, 156, 158, 160, 162, 164, 166, 168, 170, 172, 174, 176, 178, 180, 182, 184, 186, 188, 190, 192, 194, 196, 198, 200, 202, 204, 206, 208, 210, 212, 214, 216, 218, 220, 222, 224, 226,
 228, 230, 232, 234, 236, 238, 240, 242, 244, 246, 248, 250, 252, 254, 258, 260, 262, 264, 266, 268, 270, 272, 274, 276, 278, 280, 282, 284, 286, 288, 290, 292, 294, 296, 298, 300, 302, 304, 306, 308, 310, 312, 314, 316, 318, 320, 322, 324, 326, 328, 330,
 332, 334, 336, 338, 340, 342, 344, 346, 348, 350, 352, 354, 356, 358, 360, 362, 364, 366, 368, 370, 372, 374, 376, 378, 380, 382, 384, 386, 388, 390, 392, 394, 396, 398, 400, 402, 404, 406, 408, 410, 412, 414, 416, 418, 420, 422, 424, 426, 428, 430, 432,
 434, 436, 438, 440, 442, 444, 446, 448, 450, 452, 454, 456, 458, 460, 462, 464, 466, 468, 470, 472, 474, 476, 478, 480, 482, 484, 486, 488, 490, 492, 494, 496, 498, 500, 502, 504, 506, 508, 510, 514, 516, 518, 520, 522, 524, 526, 528, 530, 532, 534, 536,
 538, 540, 542, 544, 546, 548, 550, 552, 554, 556, 558, 560, 562, 564, 566, 568, 570, 572, 574, 576, 578, 580, 582, 584, 586, 588, 590, 592, 594, 596, 598, 600, 602, 604, 606, 608, 610, 612, 614, 616, 618, 620, 622, 624, 626, 628, 630, 632, 634, 636, 638,
 640, 642, 644, 646, 648, 650, 652, 654, 656, 658, 660, 662, 664, 666, 668, 670, 672, 674, 676, 678, 680, 682, 684, 686, 688, 690, 692, 694, 696, 698, 700, 702, 704, 706, 708, 710, 712, 714, 716, 718, 720, 722, 724, 726, 728, 730, 732, 734, 736, 738, 740,
 742, 744, 746, 748, 750, 752, 754, 756, 758, 760, 762, 764, 766, 768, 770, 772, 774, 776, 778, 780, 782, 784, 786, 788, 790, 792, 794, 796, 798, 800, 802, 804, 806, 808, 810, 812, 814, 816, 818, 820, 822, 824, 826, 828, 830, 832, 834, 836, 838, 840, 842,
 844, 846, 848, 850, 852, 854, 856, 858, 860, 862, 864, 866, 868, 870, 872, 874, 876, 878, 880, 882, 884, 886, 888, 890, 892, 894, 896, 898, 900, 902, 904, 906, 908, 910, 912, 914, 916, 918, 920, 922, 924, 926, 928, 930, 932, 934, 936, 938, 940, 942, 944,
 946, 948, 950, 952, 954, 956, 958, 960, 962, 964, 966, 968, 970, 972, 974, 976, 978, 980, 982, 984, 986, 988, 990, 992, 994, 996, 998, 1000, 1002, 1004, 1006, 1008, 1010, 1012, 1014, 1016, 1018, 1020, 1022, 1026, 1028, 1030, 1032, 1034, 1036, 1038, 1040,
 1042, 1044, 1046, 1048, 1050, 1052, 1054, 1056, 1058, 1060, 1062, 1064, 1066, 1068, 1070, 1072, 1074, 1076, 1078, 1080, 1082, 1084, 1086, 1088, 1090, 1092, 1094, 1096, 1098, 1100, 1102, 1104, 1106, 1108, 1110, 1112, 1114, 1116, 1118, 1120, 1122, 1124, 1126,
 1128, 1130, 1132, 1134, 1136, 1138, 1140, 1142, 1144, 1146, 1148, 1150, 1152, 1154, 1156, 1158, 1160, 1162, 1164, 1166, 1168, 1170, 1172, 1174, 1176, 1178, 1180, 1182, 1184, 1186, 1188, 1190, 1192, 1194, 1196, 1198}
```

5) Write a Python program to remove the duplicates from a list, and find the largest and smallest elements from the list.

```python
n = int(input('No. of Elements: '))
l = [0]*n
for i in range(n):
    l[i] = int(input('Element {}: '.format(i+1)))
print('Original List: ',l)
l = list(set(l))
print('List without Duplicates:',l)
print('Largest Value:',max(l))
print('Smallest Value:',min(l))
```

Rough Work ×

```
C:\Users\Yuvi\PycharmProjects\pythonProject\venv\Scri
 Work.py"
No. of Elements: 7
Element 1: 3
Element 2: 7
Element 3: 7
Element 4: 3
Element 5: 73
Element 6: 3
Element 7: 37
Original List:  [3, 7, 7, 3, 73, 3, 37]
List without Duplicates: [73, 3, 37, 7]
Largest Value: 73
Smallest Value: 3
```

6) Create 3 tuples a, b, and c such that a contains odd numbers from 1 to 10, b contains even numbers from 1 to 10, and c contains numbers divisible by both 2 and 3 from 1 to 20. Convert these tuples into sets, s1, s2, and s3, and perform the following operations:

a) Union of s1, s2, and s3

b) Check if s1 and s2 have any elements in common, and if they do, display the elements

```python
a = tuple([x for x in range(1,10,2)])
b = tuple([x for x in range(2,11,2)])
c = tuple([x for x in range(1,21) if x%6==0])
print(f'a = {a}\nb = {b}\nc = {c}')
s1, s2, s3 = set(a), set(b), set(c)
print(f'Union: {s1|s2|s3}')
if len(s1&s2) != 0:
    print(f'Common Elements of a and b: {s1&s2}')
else:
    print('No common elements.')
```

Rough Work ×

```
C:\Users\Yuvi\PycharmProjects\pythonProject\venv\Sc
 "C:/Users/Yuvi/PycharmProjects/pythonProject/venv/
a = (1, 3, 5, 7, 9)
b = (2, 4, 6, 8, 10)
c = (6, 12, 18)
Union: {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 12, 18}
No common elements.
```

7) Write a Python program to find a transpose of any m x n matrix.

```python
print('Creating Matrix...')
rows_matrix = int(input('No. of Rows: '))
columns_matrix = int(input('No. of Columns: '))
matrix = [[0]*columns_matrix for i in range(rows_matrix)]
transpose = [[0] * rows_matrix for i in range(columns_matrix)]
for i in range(rows_matrix):
    for j in range(columns_matrix):
        matrix[i][j] = int(input('Row {} Column {}: '.format(i,j)))
        transpose[j][i] = matrix[i][j]

print('Matrix:\n')
for row in matrix:
    print(row)
print('\nTranspose:\n')
for row in transpose:
    print(row)
```

```
Creating Matrix...
No. of Rows: 2
No. of Columns: 3
Row 0 Column 0: 1
Row 0 Column 1: 2
Row 0 Column 2: 3
Row 1 Column 0: 4
Row 1 Column 1: 5
Row 1 Column 2: 6
Matrix:

[1, 2, 3]
[4, 5, 6]

Transpose:

[1, 4]
[2, 5]
[3, 6]
```

8) Write a Python program to perform matrix multiplication on two given matrices.

```
print('Creating Matrix M1...')
rows_m1 = int(input('No. of Rows: '))
columns_m1 = int(input('No. of Columns: '))
m1 = [[0]*columns_m1 for i in range(rows_m1)]
for i in range(rows_m1):
    for j in range(columns_m1):
        m1[i][j] = int(input('Row {} Column {}: '.format(i,j)))
print('Creating Matrix M2...')
rows_m2 = int(input('No. of Rows: '))
columns_m2 = int(input('No. of Columns: '))
m2 = [[0]*columns_m2 for i in range(rows_m2)]
for i in range(rows_m2):
    for j in range(columns_m2):
        m2[i][j] = int(input('Row {} Column {}: '.format(i,j)))
if columns_m1 == rows_m2:
    result = [[0]*columns_m2 for i in range(rows_m1)]
    for i in range(rows_m1):
        for j in range(columns_m2):
            for k in range(columns_m1):  # or rows_m2
                result[i][j] += m1[i][k] * m2[k][j]
    print('\nMatrix 1\n')
    for row in m1:
        print(row)
    print('\nMatrix 2\n')
    for row in m2:
        print(row)
    print('\nM1 x M2\n')
    for row in result:
        print(row)
else:
    print('\nSorry, these two matrices can\'t be multiplied.')
```

```
Creating Matrix M1...
No. of Rows: 3
No. of Columns: 2
Row 0 Column 0: 1
Row 0 Column 1: 2
Row 1 Column 0: 3
Row 1 Column 1: 4
Row 2 Column 0: 5
Row 2 Column 1: 6
Creating Matrix M2...
No. of Rows: 2
No. of Columns: 3
Row 0 Column 0: 1
Row 0 Column 1: 2
Row 0 Column 2: 3
Row 1 Column 0: 4
Row 1 Column 1: 5
Row 1 Column 2: 6

Matrix 1

[1, 2]
[3, 4]
[5, 6]

Matrix 2

[1, 2, 3]
[4, 5, 6]

M1 x M2

[9, 12, 15]
[19, 26, 33]
[29, 40, 51]
```

```
Creating Matrix M1...
No. of Rows: 2
No. of Columns: 2
Row 0 Column 0: 1
Row 0 Column 1: 2
Row 1 Column 0: 3
Row 1 Column 1: 4
Creating Matrix M2...
No. of Rows: 3
No. of Columns: 2
Row 0 Column 0: 1
Row 0 Column 1: 2
Row 1 Column 0: 3
Row 1 Column 1: 4
Row 2 Column 0: 5
Row 2 Column 1: 6

Sorry, these two matrices can't be multiplied.
```